

Министерство науки и высшего образования Российской Федерации

Федеральное государственное автономное образовательное учреждение
высшего образования
«СЕВЕРО-КАВКАЗСКИЙ ФЕДЕРАЛЬНЫЙ УНИВЕРСИТЕТ»

Институт перспективной инженерии
Департамента цифровых, робототехнических систем и электроники

ОТЧЕТ
ПО ЛАБОРАТОРНОЙ РАБОТЕ №1
дисциплины «Объектно-ориентированное программирование»
Вариант 12

Выполнила:
Коробка В.А.,
3 курс, группа ИВТ-б-о-24-1,
09.03.01 «Информатика и
вычислительная техника»,
направленность (профиль)
«Программное обеспечение средств
вычислительной техники и
автоматизированных систем», очная
форма обучения

(подпись)

Руководитель практики:
Воронкин Роман Александрович,
доцент департамента цифровых,
робототехнических систем и
электроники института перспективной
инженерии

(подпись)

Отчет защищен с оценкой _____ Дата защиты _____
Ставрополь, 2026 г.

Тема: Элементы объектно-ориентированного программирования в языке Python

Цель: приобретение навыков по работе с классами и объектами при написании программ с помощью языка программирования Python версии 3.x.

Порядок выполнения работы

В ходе лабораторной работы был изучен теоретический материал, создан и клонирован публичный репозиторий на GitHub с лицензией MIT. В созданном проекте были проработаны все примеры лабораторной работы.

Условие индивидуального задания по варианту 12:

Парой называется класс с двумя полями, которые обычно имеют имена `first` и `second`. Требуется реализовать тип данных с помощью такого класса. Во всех заданиях обязательно должны присутствовать: метод инициализации `__init__` ; метод должен контролировать значения аргументов на корректность; ввод с клавиатуры `read` ; вывод на экран `display` .

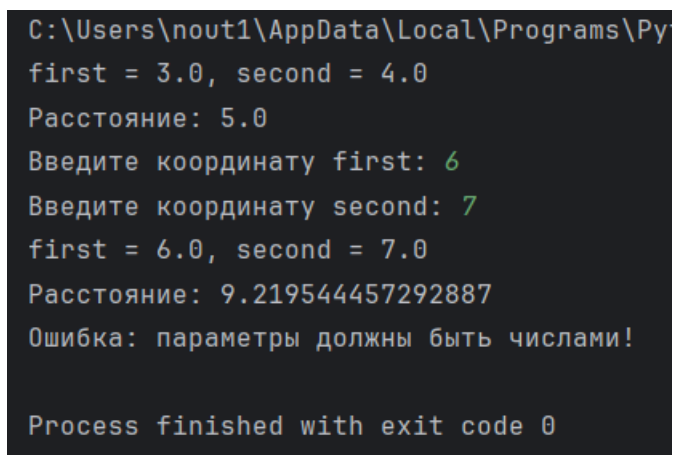
Реализовать внешнюю функцию с именем `make_тип()` , где `тип` — тип реализуемой структуры. Функция должна получать в качестве аргументов значения для полей структуры и возвращать структуру требуемого типа. При передаче ошибочных параметров следует выводить сообщение и заканчивать работу.

В раздел программы, начинающийся после инструкции `if __name__ == '__main__':` добавить код, демонстрирующий возможности разработанного класса.

Поле `first` — дробное число, координата точки на плоскости; поле `second` — дробное число, координата точки на плоскости. Реализовать метод `distance()` — расстояние точки от начала координат.

Был реализован класс `Pair`, представляющий точку на плоскости с двумя дробными координатами `first` и `second`. Класс содержит метод инициализации `__init__`, который проверяет корректность передаваемых аргументов с помощью

функции `isinstance`. Метод `read` обеспечивает ввод координат с клавиатуры, метод `display` выводит текущие значения полей на экран с использованием форматирования строк, метод `distance` вычисляет расстояние от точки до начала координат по формуле. Дополнительно реализована внешняя функция `make_pair`, которая принимает два аргумента, проверяет их на принадлежность к числовым типам и возвращает созданный объект класса `Pair`. При передаче некорректных параметров функция выводит сообщение об ошибке и завершает работу программы.



```
C:\Users\nout1\AppData\Local\Programs\Python\Python39\python.exe
first = 3.0, second = 4.0
Расстояние: 5.0
Введите координату first: 6
Введите координату second: 7
first = 6.0, second = 7.0
Расстояние: 9.219544457292887
Ошибка: параметры должны быть числами!

Process finished with exit code 0
```

Рисунок 1. Результат выполнения индивидуального задания 1

Полный код задания:

```
import math
```

```
class Pair:
```

```
    def __init__(self, first, second):
```

```
        if isinstance(first, (int, float)) and isinstance(second, (int, float)):
```

```
            self.first = float(first)
```

```
            self.second = float(second)
```

```
        else:
```

```
            raise ValueError
```

```

def read(self):
    self.first = float(input("Введите координату first: "))
    self.second = float(input("Введите координату second: "))

def display(self):
    print("first = {}, second = {}".format(self.first, self.second))

def distance(self):
    return math.sqrt(self.first ** 2 + self.second ** 2)

def make_pair(first, second):
    if isinstance(first, (int, float)) and isinstance(second, (int, float)):
        return Pair(first, second)
    else:
        print("Ошибка: параметры должны быть числами!")
        exit()

if __name__ == '__main__':
    p1 = make_pair(3, 4)
    p1.display()
    print("Расстояние: {}".format(p1.distance()))

    p2 = Pair(0, 0)
    p2.read()
    p2.display()
    print("Расстояние: {}".format(p2.distance()))

```

```
p3 = make_pair("abc", 5)
```

Условие индивидуального задания 2:

Составить программу с использованием классов и объектов для решения задачи. Во всех заданиях, помимо указанных в задании операций, обязательно должны быть реализованы следующие методы: метод инициализации `__init__` ; ввод с клавиатуры `read` ; вывод на экран `display` .

В раздел программы, начинающийся после инструкции `if __name__ == '__main__':` добавить код, демонстрирующий возможности разработанного класса.

12. Создать класс `Fraction` для работы с дробными числами. Число должно быть представлено двумя целочисленными полями: целая часть и дробная часть. Реализовать арифметические операции сложения, вычитания, умножения и операции сравнения.

Был реализован класс `Fraction` для работы с обыкновенными дробями. Дробь хранится в двух целочисленных полях: `chislitel` и `znamenatel`. Метод `__init__` проверяет, что переданные значения являются целыми числами, а знаменатель не равен нулю. Метод `reduce` сокращает дробь с помощью функции `math.gcd` и переносит знак в числитель, если знаменатель отрицательный. Метод `read` обеспечивает ввод числителя и знаменателя с клавиатуры, а метод `display` выводит дробь в формате числитель/знаменатель. Для класса созданы арифметические операции сложения, вычитания и умножения с помощью методов `__add__`, `__sub__` и `__mul__`. Также реализованы операции сравнения: равенство, неравенство, меньше, больше, меньше или равно и больше или равно. В блоке `if __name__ == '__main__':` создаются две дроби, демонстрируются арифметические операции и сравнения, после чего выполняется ввод третьей дроби с клавиатуры и её вывод на экран.

```
C:\Users\nout1\AppData\Loc
f1 = 1/2
f2 = 3/4
Сложение (f1 + f2): 5/4
Вычитание (f1 - f2): -1/4
Умножение (f1 * f2): 3/8
f1 == f2: False
f1 != f2: True
f1 < f2: True
f1 > f2: False
f1 <= f2: True
f1 >= f2: False
Введите числитель: 6
Введите знаменатель: 7
f3 = 6/7
```

Рисунок 2. Результат выполнения задания 2

Полный код задания:

```
import math
```

```
class Fraction:
```

```
    def __init__(self, chislitel, znamenatel):
```

```
        if not isinstance(chislitel, int) or not isinstance(znamenatel, int):
```

```
            raise ValueError
```

```
        if znamenatel == 0:
```

```
            raise ValueError
```

```
        self.chislitel = chislitel
```

```
        self.znamenatel = znamenatel
```

```
        self.reduce()
```

```
    def reduce(self):
```

```
nod = math.gcd(abs(self.chislitel), abs(self.znamenatel))
self.chislitel //= nod
self.znamenatel //= nod
if self.znamenatel < 0:
    self.chislitel = -self.chislitel
    self.znamenatel = -self.znamenatel
```

```
def read(self):
    self.chislitel = int(input("Введите числитель: "))
    self.znamenatel = int(input("Введите знаменатель: "))
    if self.znamenatel == 0:
        print("Ошибка: знаменатель не может быть равен нулю!")
        exit()
    self.reduce()
```

```
def display(self):
    print("{} / {}".format(self.chislitel, self.znamenatel))
```

Сложение

```
def __add__(self, other):
    c = self.chislitel * other.znamenatel + other.chislitel * self.znamenatel
    z = self.znamenatel * other.znamenatel
    return Fraction(c, z)
```

Вычитание

```
def __sub__(self, other):
    c = self.chislitel * other.znamenatel - other.chislitel * self.znamenatel
    z = self.znamenatel * other.znamenatel
```

```
return Fraction(c, z)
```

```
# Умножение
```

```
def __mul__(self, other):
```

```
    c = self.chislitel * other.chislitel
```

```
    z = self.znamenatel * other.znamenatel
```

```
    return Fraction(c, z)
```

```
# Сравнения
```

```
def __eq__(self, other):
```

```
    return self.chislitel * other.znamenatel == other.chislitel * self.znamenatel
```

```
def __ne__(self, other):
```

```
    return not self.__eq__(other)
```

```
def __lt__(self, other):
```

```
    return self.chislitel * other.znamenatel < other.chislitel * self.znamenatel
```

```
def __gt__(self, other):
```

```
    return self.chislitel * other.znamenatel > other.chislitel * self.znamenatel
```

```
def __le__(self, other):
```

```
    return self.__lt__(other) or self.__eq__(other)
```

```
def __ge__(self, other):
```

```
    return self.__gt__(other) or self.__eq__(other)
```



```
if __name__ == '__main__':  
    f1 = Fraction(1, 2)  
    f2 = Fraction(3, 4)  
  
    print("f1 = ", end="")  
    f1.display()  
    print("f2 = ", end="")  
    f2.display()  
  
    print("Сложение (f1 + f2): ", end="")  
    (f1 + f2).display()  
  
    print("Вычитание (f1 - f2): ", end="")  
    (f1 - f2).display()  
  
    print("Умножение (f1 * f2): ", end="")  
    (f1 * f2).display()  
  
    print("f1 == f2: {}".format(f1 == f2))  
    print("f1 != f2: {}".format(f1 != f2))  
    print("f1 < f2: {}".format(f1 < f2))  
    print("f1 > f2: {}".format(f1 > f2))  
    print("f1 <= f2: {}".format(f1 <= f2))  
    print("f1 >= f2: {}".format(f1 >= f2))  
  
    f3 = Fraction(1, 1)  
    f3.read()  
    print("f3 = ", end="")
```

f3.display()

Ответы на контрольные вопросы

1. Как осуществляется объявление класса в языке Python?

Классы объявляются с помощью ключевого слова `class` и имени класса. Как правило, имя класса начинается с заглавной буквы и обычно является существительным или словосочетанием.

2. Чем атрибуты класса отличаются от атрибутов экземпляра?

Атрибуты класса являются общими для всех объектов класса, а атрибуты экземпляра специфическими для каждого экземпляра. Атрибуты класса определяются внутри класса, но вне каких-либо методов, а атрибуты экземпляра обычно определяются в методах, чаще всего в `__init__`.

3. Каково назначение методов класса?

Методы определяют функциональность объектов, принадлежащих конкретному классу.

4. Для чего предназначен метод `__init__()` класса?

Метод `__init__()` является конструктором. Конструкторы – это концепция объектно-ориентированного программирования. Метод `__init__()` указывает, какие атрибуты будут у экземпляров нашего класса.

5. Каково предназначение `self`?

Аргумент `self` представляет конкретный экземпляр класса и позволяет нам получить доступ к его атрибутам и методам.

6. Как добавить атрибуты в класс?

Атрибуты класса добавляются путём присваивания значений внутри тела класса, но вне методов. Атрибуты экземпляра добавляются внутри методов, чаще всего в `__init__`, через обращение к `self`.

7. Как осуществляется управление доступом к методам и атрибутам в языке Python?

Атрибуты и методы, имя которых начинается с одного подчёркивания, считаются защищёнными и не предназначены для использования извне. Атрибуты с двойным подчёркиванием являются приватными и подвергаются искажению имён. Для контролируемого доступа к приватным атрибутам используются декораторы `@property` и `@имя_атрибута.setter`.

8. Каково назначение функции `isinstance`?

Функция `isinstance` проверяет, принадлежит ли объект указанному типу или классу.

Вывод: В ходе выполнения лабораторной работы были изучены основные принципы объектно-ориентированного программирования в языке Python и приобретены практические навыки работы с классами и объектами. В процессе выполнения индивидуальных заданий были реализованы классы `Pair` и `Fraction`, в которых использовались методы инициализации с валидацией данных, ввод и вывод информации, а также перегрузка арифметических операторов и операторов сравнения.